

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL2- DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:	U.Prabhavathi	Reg.No.:	192472067
Department:	CSE-AI	Date:	01-08-26

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.
2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.
3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.
4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.
5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

1. Signed Integer Addition Using 2's Complement – Overflow Detection

Problem

A program performs signed integer addition using 8-bit 2's complement, but gives incorrect results when adding large positive numbers such as +120 and +50.

Step 1: Represent the Numbers in 8-bit Binary

+120

01111000

+50

00110010

Step 2: Perform Binary Addition

01111000 (+120)
+ 00110010 (+50)

10101010

Result obtained:

10101010

In 2's complement,

10101010 = -86

Actual mathematical result:

120 + 50 = 170

Since an 8-bit signed integer can only represent:

-128 to +127

170 cannot be represented.

Hence,

Overflow has occurred.

Step 3: Why Overflow Happens

Overflow occurs when

- Two positive numbers produce a negative result.
- Two negative numbers produce a positive result.

Here,

Positive + Positive = Negative

$120 + 50 = -86$ (incorrect)

Therefore,

Overflow is detected.

Step 4: Overflow Detection Logic

Overflow occurs if

Carry into MSB $\neq$ Carry out of MSB

or

$\text{Sign}(A) = \text{Sign}(B)$

AND

$\text{Sign}(\text{Result}) \neq \text{Sign}(A)$

Example:

A Sign = 0

B Sign = 0

Result Sign = 1

Overflow = YES

Step 5: Solution

Use overflow detection circuitry.

Pseudo code:

```
if ((A>0 && B>0 && Result<0) ||  
    (A<0 && B<0 && Result>=0))  
{  
    Overflow = TRUE;  
}
```

Or

Increase the word size

8-bit $\rightarrow$ 16-bit

so larger numbers can be represented.

Final Answer

The incorrect result occurs because the sum exceeds the range of an 8-bit signed integer. Overflow is detected by comparing the carry into and carry out of the sign bit or by checking sign changes. Using overflow detection logic or increasing the register size prevents incorrect arithmetic results.

2. Ripple Carry Adder Performance Bottleneck

Problem

A processor uses a Ripple Carry Adder (RCA), causing delays during arithmetic operations.

Working of Ripple Carry Adder

Each full adder waits for the previous carry.

Example (4-bit)

A3 A2 A1 A0
B3 B2 B1 B0

FA0 → FA1 → FA2 → FA3

Carry propagation

C0 → C1 → C2 → C3 → C4

Each stage must wait.

Delay Analysis

If one full adder takes

2 ns

For a 4-bit RCA

Total Delay

=4 × 2

=8 ns

For a 32-bit adder

32 × 2

=64 ns

Delay increases linearly.

Problem

Carry propagation becomes the performance bottleneck.

Real-time systems require faster execution.

Solution: Carry Look-Ahead Adder (CLA)

Instead of waiting,

CLA predicts carries using

Generate

$$G_i = A_i \times B_i$$

Propagate

$$P_i = A_i \oplus B_i$$

Carry equations

$$C_1 = G_0 + P_0C_0$$

$$C_2 = G_1 + P_1G_0 + P_1P_0C_0$$

$$C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$$

All carries are calculated simultaneously.

Advantages

- No waiting for previous carry
- Parallel carry computation
- Faster execution
- Suitable for high-speed processors

Comparison

Ripple Carry Adder	Carry Look-Ahead Adder
Carry propagates sequentially	Carry generated in parallel
Slow	Fast
Delay increases with bits	Much lower delay
Simple hardware	Slightly complex hardware

Final Answer

The delay occurs because each carry must propagate through every full adder. A Carry Look-Ahead Adder computes carries in parallel using generate and propagate signals, significantly reducing delay and improving processor performance.

3. Booth's Algorithm Debugging

Problem

Booth's multiplication produces incorrect results for negative operands.

Example

Multiply

$$-5 \times 3$$

Binary (4-bit)

Multiplicand (M)

-5

0101

2's complement

1011

Multiplier (Q)

3

0011

Initialize

A = 0000

Q = 0011

Q-1 = 0

Booth's Rules

Q0 Q-1	Operation
00	No operation
11	No operation
01	Add M
10	Subtract M

Steps

Check

Q0 Q-1

Perform

- Add
- Subtract
- Arithmetic right shift

Repeat for all bits.

Common Errors

Error 1

Wrong bit-pair checking

Using

Q1 Q0

instead of

Q0 Q-1

Error 2

Logical shift instead of Arithmetic shift

Wrong

00111010

Correct

11111010

Sign bit must be preserved.

Error 3

Improper sign extension

Negative numbers lose sign.

Error 4

Incorrect subtraction

Subtracting M directly instead of

$A + (-M)$

Corrections

Correct bit-pair checking

Arithmetic right shift

Proper sign extension

Correct 2's complement subtraction

Final Answer

Incorrect multiplication results occur due to errors in bit-pair evaluation, arithmetic shifting, sign extension, or subtraction. Proper implementation of Booth's rules, arithmetic right shifts, and correct handling of signed numbers ensures accurate multiplication.

4. Restoring Division Algorithm

Problem

The restoring division algorithm repeatedly restores the partial remainder, increasing execution time.

Restoring Division Steps

For each bit

1. Shift left
2. Subtract divisor
3. Check sign

If remainder negative

Restore

$R = R + \text{Divisor}$

Set quotient bit

0

Else

Keep remainder

Set quotient bit

1

Problem

When subtraction gives a negative remainder,

Subtract



Negative



Restore



Repeat

Extra addition increases execution time.

Example

Dividend =13

Divisor =3

Many cycles require

Subtract

Restore

Subtract

Restore

More clock cycles are needed.

Optimized Solution

Non-Restoring Division

Instead of restoring immediately,

If

$R < 0$

Next cycle

Add divisor

Else

Subtract divisor

Only one correction is made at the end.

Advantages

- Eliminates repeated restoration
- Fewer arithmetic operations
- Reduced clock cycles

- Faster execution
- Better processor efficiency

Comparison

Restoring	Non-Restoring
Frequent restoration	No immediate restoration
More additions	Fewer additions
Slow	Fast
Higher execution time	Lower execution time

Final Answer

The inefficiency is caused by repeated restoration whenever the remainder becomes negative. The non-restoring division algorithm avoids unnecessary restoration and performs correction only when needed, reducing arithmetic operations and improving execution speed.

5. IEEE 754 Floating-Point Addition Accuracy

Problem

A scientific application using IEEE 754 single precision (32-bit) gives inaccurate results when adding a very small number to a very large number.

Example

Large number

100000000

Small number

1

Expected

100000001

Computed (Single Precision)

100000000

The small value is lost.

IEEE 754 Single Precision Format

1 Sign bit

8 Exponent bits

23 Mantissa bits

Step 1: Exponent Alignment

Before addition,

Both exponents must match.

Large exponent

100000000

Small exponent

1

The mantissa of the smaller number is shifted right many times.

Eventually,

0.000000000...

The significant bits disappear.

Step 2: Addition

Only the large number contributes significantly.

100000000

+

0

=

100000000

Step 3: Normalization

Result is normalized.

No improvement in lost precision.

Step 4: Rounding Error

Mantissa has only

23 bits

Extra bits are rounded.

Small values disappear.

Causes of Inaccuracy

- Exponent alignment shifts away small values
- Limited mantissa precision
- Rounding during normalization
- Loss of significant digits (catastrophic cancellation in some operations)

Optimization Techniques

1. Use Double Precision

64-bit

1 Sign

11 Exponent

52 Mantissa

Greater precision reduces rounding errors.

2. Rearrange Calculations

Add numbers of similar magnitude first.

Example

Wrong

$100000000 + 1 + 2$

Better

$1 + 2 + 100000000$

3. Use Numerically Stable Algorithms

Techniques such as compensated summation (e.g., Kahan summation) reduce accumulated rounding error.

4. Avoid Unnecessary Conversions

Keep calculations in the same precision whenever possible.

Final Answer

The loss of accuracy occurs because exponent alignment shifts the smaller operand's mantissa, causing precision loss and rounding errors in single precision. Using double precision, adding similar-sized numbers first, and applying numerically stable algorithms significantly improve the accuracy of floating-point computations.

Code of Conduct:

I U.Prabhavathi_/192472067_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

U.Prabhavathi

Signature of the student:

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided